

Tema 8.

Clasificación temática de textos

Palabras clave, vectores TF, similitud del coseno y clasificación textual con Python

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Dr.^a Blanca Hernández Pardo

Índice de contenidos

Contexto profesional.....	1
Encargo: organizar un archivo textual heterogéneo.....	1
El caso de trabajo.....	1
Qué vas a aprender en esta unidad.....	2
Mapa conceptual de la unidad.....	2
1. Marco teórico.....	3
1.1. Qué es la clasificación temática de textos.....	3
1.2. Clasificación, palabras clave e indexación.....	3
1.3. Representar textos como vectores.....	4
1.4. TF: frecuencia de términos.....	4
1.5. IDF y TF-IDF.....	4
1.6. Similitud léxica y similitud coseno.....	5
1.7. Prototipos temáticos y clasificación por cercanía.....	6
1.8. Preprocesamiento lingüístico con spaCy.....	6
1.9. Explicabilidad: evidencias léxicas.....	7
1.10. Clasificación temática frente a modelos neuronales.....	7
1.11. Límites lingüísticos del método.....	8
1.12. Qué significa «clasificar con responsabilidad».....	9
2. Clasificación temática explicable con Python.....	9
2.1. Preparar el entorno.....	9
2.2. Textos para clasificar.....	10
2.3. Perfiles temáticos mediante palabras clave.....	10
2.4. Preprocesamiento con spaCy.....	11
2.5. Vectorización TF con Counter.....	11
2.6. Similitud coseno en Python.....	11
2.7. Construir prototipos temáticos.....	12
2.8. Evidencias léxicas.....	12
2.9. Clasificar un texto.....	13
2.10. Clasificar varios textos y crear una tabla.....	13
2.11. Informe de clasificación.....	14
2.12. Dónde entraría TF-IDF.....	14
2.13. Cómo leer los resultados antes de la práctica.....	15
Errores frecuentes de aprendizaje.....	15
Síntesis final.....	16
Cierre narrativo.....	16
Para saber más: Preparación previa a la práctica.....	16
Referencias.....	17

Título narrativo

Clasificar textos con evidencias: un sistema que, además de decidir, muestra sus razones.

Contexto profesional

Encargo: organizar un archivo textual heterogéneo

Imagina que te incorporas como analista júnior de PLN al **Centro Atlas de Documentación Textual**, una institución pública que gestiona colecciones digitales procedentes de administraciones, bibliotecas, servicios técnicos, medios culturales y repositorios educativos. Cada semana llegan documentos muy distintos: fragmentos jurídicos, manuales técnicos, textos divulgativos, materiales literarios y documentos híbridos que no siempre encajan limpiamente en una sola categoría.

El equipo de archivo necesita clasificar esos textos antes de indexarlos, derivarlos a especialistas o incorporarlos a una base documental. El problema va más allá de asignar una etiqueta, porque hay que justificar la decisión: si un texto se archiva como jurídico, técnico, divulgativo o literario, la persona responsable debe poder explicar qué señales lingüísticas han llevado a esa clasificación.

La institución no quiere empezar con un modelo neuronal opaco: no dispone de miles de textos etiquetados y trabaja con documentos sensibles en los que las decisiones deben poder revisarse. Por eso encarga un sistema sencillo, auditable y pedagógicamente transparente: un clasificador temático basado en palabras clave, vectores simples de frecuencia y similitud coseno.

El caso de trabajo

El flujo profesional se formula así:

1. Definir varias clases temáticas: jurídico, técnico, divulgativo y literario.
2. Construir un perfil o prototipo de cada clase mediante palabras clave.
3. Preprocesar los textos con criterios lingüísticos.
4. Convertir cada texto en un vector de frecuencias.
5. Comparar el vector del texto con cada prototipo.
6. Asignar la clase con mayor similitud.
7. Mostrar evidencias léxicas: qué palabras compartidas explican la decisión.

El valor del sistema va más allá de la etiqueta final: permite decir «este texto se parece más al prototipo jurídico porque comparte lemas como *ley*, *contrato* u *obligación*», una explicación que puede discutirse, corregirse y servir para mejorar los prototipos.

En esta unidad aprenderás a construir ese clasificador explicable. Más que competir con los modelos de aprendizaje profundo, se busca entender una de las ideas fundamentales del PLN clásico: un texto puede representarse como un vector, y dos textos pueden compararse por la orientación de esos vectores.

Qué vas a aprender en esta unidad

Al finalizar esta unidad, deberías ser capaz de:

- explicar qué es la clasificación temática de textos;
- distinguir entre clasificación, etiquetado, indexación y agrupación temática;
- entender qué papel cumplen las palabras clave como señales interpretables;
- representar un texto como bolsa de palabras;
- explicar qué es un vector simple de frecuencia o TF;
- comprender por qué TF cuenta palabras pero no mide por sí solo la especificidad;
- explicar el funcionamiento de IDF y TF-IDF;
- interpretar la similitud coseno como medida de parecido léxico entre vectores;
- comprender qué es un prototipo temático;
- explicar cómo se construyen prototipos por clase a partir de palabras clave;
- aplicar tokenización, lematización y eliminación de *stopwords* con spaCy;
- entender cómo `Counter` representa un vector TF;
- seguir paso a paso una función de similitud coseno en Python;
- justificar una decisión de clasificación mediante evidencias léxicas;
- reconocer los límites del enfoque en textos híbridos, breves, irónicos o con sinonimia.

Mapa conceptual de la unidad

Tabla 1. Mapa conceptual de la unidad: conceptos y operaciones en Python

Bloque	Concepto	Operación en Python	Resultado
Clases temáticas	Categorías de archivo o análisis	Diccionario de etiquetas	Jurídico, técnico, divulgativo, literario
Palabras clave	Señales léxicas de cada clase	<code>class_keywords</code>	Prototipos interpretables
Preprocesamiento	Preparar el texto lingüísticamente	spaCy: tokenizar, lematizar, filtrar	Lista de lemas
TF	Frecuencia de términos	<code>Counter(tokens)</code>	Vector simple del texto
TF-IDF	Frecuencia ponderada por especificidad	TF por IDF	Vector más sensible a palabras distintivas
Coseno	Parecido entre vectores	Producto escalar y normas	Puntuación de similitud
Prototipo	Perfil vectorial de una clase	Palabras clave preprocesadas y vectorizadas	Vector por temática
Clasificación	Elección de la clase más similar	<code>max(scores, key=scores.get)</code>	Etiqueta predicha
Evidencias	Rasgos que justifican la decisión	Intersección texto/prototipo	Lemas compartidos
Informe	Salida legible y auditable	<code>DataFrame</code> y <code>print_report</code>	Tabla y explicación

1. Marco teórico

1.1. Qué es la clasificación temática de textos

La clasificación temática consiste en asignar a un texto una etiqueta que resume su tema, dominio o tipo de contenido. En esta unidad las etiquetas son jurídico, técnico, divulgativo y literario, pero el mismo enfoque podría adaptarse a noticias, reseñas, documentos administrativos, comunicaciones científicas o textos patrimoniales.

La clasificación de textos es una tarea central del PLN. Sebastiani (2002) la describe como la asignación automática de documentos a categorías predefinidas, una tarea que creció con la disponibilidad masiva de documentos digitales, y Kowsari *et al.* (2019) y Gasparetto *et al.* (2022) muestran que se ha abordado con métodos muy variados: reglas, bolsas de palabras, TF-IDF, Naive Bayes, SVM, árboles, redes neuronales y transformadores.

La unidad se coloca deliberadamente en el lado interpretable de esa tradición: no entrena un modelo complejo, sino que usa clases definidas por personas, palabras clave visibles y una medida de similitud que podría calcularse a mano si fuera necesario. Así, el alumnado entiende la lógica de la clasificación antes de pasar a modelos más opacos.

Una clasificación temática es una decisión operativa, y no una lectura total del texto. Un fragmento puede tener rasgos técnicos y divulgativos al mismo tiempo; un texto literario puede mencionar una ley, y un texto jurídico puede tener una introducción divulgativa. Por eso la etiqueta debe acompañarse de evidencias.

1.2. Clasificación, palabras clave e indexación

Las palabras clave son unidades léxicas que condensan información temática. En un texto jurídico pueden aparecer *ley*, *artículo*, *cláusula* u *obligación*; en un texto técnico, *dispositivo*, *puerto*, *controlador* o *configurar*, y en un texto literario, *memoria*, *noche*, *viajero* o *metáfora*. Estas palabras no tienen nada de mágico: son señales cuyo valor depende del contexto y del sistema de clases.

La extracción de palabras clave y la clasificación temática están relacionadas: las palabras clave ayudan a describir un texto, y la clasificación usa esas señales para decidir una etiqueta. Rose *et al.* (2010) presentan RAKE como un método de extracción automática de palabras clave a partir de documentos individuales, y Papagiannopoulou y Tsoumakas (2020) revisan los métodos de extracción de frases clave, desde los no supervisados hasta los supervisados y los basados en aprendizaje profundo.

En esta unidad las palabras clave no se extraen automáticamente: se definen a mano como perfiles de clase. La decisión es útil desde el punto de vista pedagógico y profesional, porque deja ver con claridad qué considera el sistema como señal temática; si una categoría falla, se puede revisar su lista de palabras clave.

Tabla 2. Palabras clave, perfiles, prototipos y evidencias

Concepto	Función	Ejemplo
Palabra clave	Señal léxica de un tema	<i>ley</i> , <i>contrato</i> , <i>controlador</i>
Perfil temático	Conjunto de señales de una clase	lista jurídica o técnica
Prototipo	Perfil convertido en vector	<i>Counter</i> de lemas
Evidencia	Coincidencia que apoya la decisión	texto y prototipo comparten <i>ley</i>

1.3. Representar textos como vectores

Una de las ideas más influyentes de la recuperación de información clásica es el **modelo de espacio vectorial**. Salton *et al.* (1975) propusieron representar los documentos como vectores, de modo que cada dimensión corresponde a un término y cada valor indica el peso de ese término en el documento. Vistas así las cosas, un texto deja de ser solo una secuencia lineal de palabras y se convierte en un punto o una dirección dentro de un espacio.

Para estudiantes de lingüística, puede imaginarse así: si el vocabulario relevante contiene *ley*, *contrato*, *puerto*, *dispositivo* y *memoria*, cada texto puede representarse con cinco posiciones. Si *ley* aparece dos veces, su posición vale 2; si *puerto* no aparece, la suya vale 0.

Tabla 3. Representación vectorial de dos textos

Término	Texto jurídico	Texto técnico
<i>ley</i>	2	0
<i>contrato</i>	1	0
<i>puerto</i>	0	1
<i>dispositivo</i>	0	2
<i>memoria</i>	0	0

Este enfoque se conoce también como bolsa de palabras (***bag-of-words***). La metáfora es útil: metemos las palabras en una bolsa, contamos cuántas hay de cada tipo y perdemos el orden, lo que simplifica mucho el problema, aunque también hace perder información sintáctica, narrativa y discursiva.

1.4. TF: frecuencia de términos

TF son las siglas de ***term frequency***, frecuencia de término. En su versión más sencilla, TF cuenta cuántas veces aparece una palabra en un texto: si un texto contiene *ley* dos veces y *contrato* una vez, su vector TF incluye *ley*: 2 y *contrato*: 1.

En Python, esta idea se implementa con `Counter`:

```
Counter(["ley", "contrato", "ley"])
```

El resultado conceptual es:

```
{"ley": 2, "contrato": 1}
```

TF es intuitivo y transparente, porque una persona puede revisar qué términos se han contado. Su problema es que favorece las palabras frecuentes dentro del texto aunque no sean necesariamente distintivas. En un corpus grande, palabras como *sistema*, *documento* o *información* pueden aparecer en muchas clases, y por eso TF suele combinarse con otras ponderaciones.

En esta unidad, el código principal usa TF porque el objetivo es comprender la mecánica de la representación vectorial: antes de ponderar, conviene entender qué se está ponderando.

1.5. IDF y TF-IDF

IDF son las siglas de ***inverse document frequency***, frecuencia inversa de documento. La idea, formulada de manera clásica por Spärck Jones (1972), es que una palabra resulta más informativa si

aparece en pocos documentos y menos si aparece en casi todos, y Robertson (2004) ofrece argumentos teóricos sobre por qué IDF funciona en recuperación de información.

La intuición es sencilla:

- si una palabra aparece en todos los documentos, ayuda poco a distinguirlos;
- si una palabra aparece en pocos documentos, puede ser más específica;
- si una palabra aparece muchas veces en un texto y poco en el resto, puede ser muy relevante para ese texto.

TF-IDF combina ambas ideas:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) * \text{IDF}(t)$$

Donde t es el término y d es el documento. En las formulaciones habituales:

$$\text{IDF}(t) = \log(N / df(t))$$

N es el número total de documentos, y $df(t)$, el número de documentos que contienen el término. Muchas implementaciones añaden suavizado para evitar divisiones problemáticas. La documentación de scikit-learn explica que `TfidfVectorizer` convierte documentos en matrices TF-IDF y que, en la práctica, combina un conteo de términos con una transformación IDF (scikit-learn developers, 2026).

En PLN8, el *pipeline* principal usa TF para mantener la explicación en el nivel más transparente, pero TF-IDF es el siguiente paso natural, porque permite que las palabras comunes entre clases pesen menos y que los términos distintivos pesen más.

1.6. Similitud léxica y similitud coseno

Una vez que los textos son vectores, necesitamos compararlos. La **similitud coseno** mide el parecido entre dos vectores por su orientación, y no por su tamaño absoluto; en términos intuitivos, dos textos son parecidos si apuntan hacia direcciones léxicas similares.

La fórmula es:

$$\text{coseno}(A, B) = (A \cdot B) / (||A|| * ||B||)$$

El numerador, $A \cdot B$, es el producto escalar, que en el código se calcula multiplicando los valores de los términos comunes y sumando. Si dos vectores no comparten términos, el producto escalar será 0.

El denominador multiplica las normas o longitudes de los vectores, una normalización que evita que un texto más largo resulte automáticamente más parecido solo porque tiene más palabras. Manning *et al.* (2008) presentan la similitud coseno como una medida central del modelo vectorial para la recuperación de información y la comparación documental.

La interpretación didáctica es:

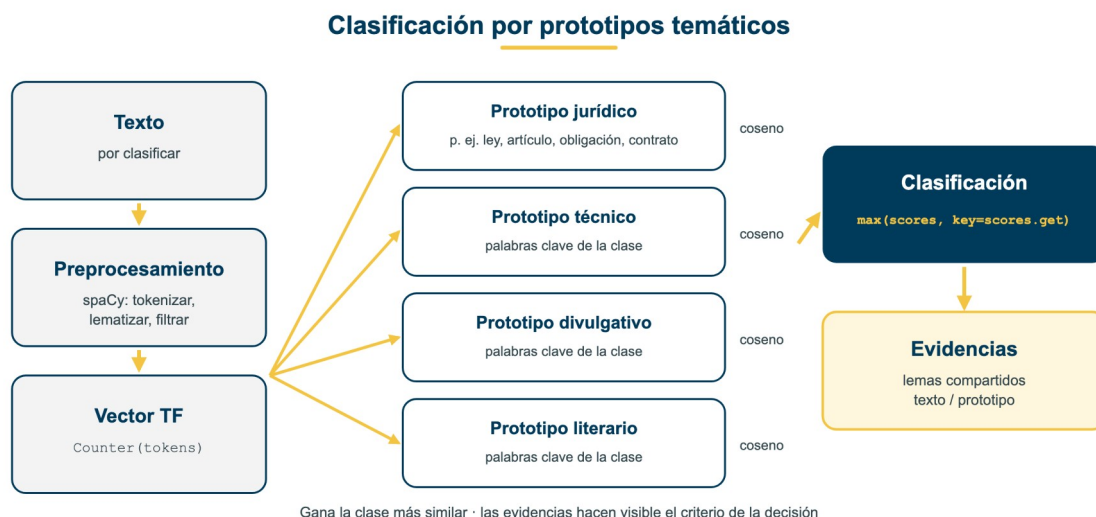
- 1: vectores muy similares;
- cercano a 0: poca coincidencia léxica;
- 0: sin términos compartidos relevantes.

En esta unidad, el coseno mide similitud léxica, y no comprensión semántica profunda: si un texto usa sinónimos que no están en el prototipo, la similitud puede ser baja aunque el tema sea correcto.

1.7. Prototipos temáticos y clasificación por cercanía

El cuaderno utiliza una estrategia basada en prototipos. Cada clase se representa mediante un conjunto de palabras clave, que se preprocesan y se convierten en un vector TF; después, cada texto se compara con cada prototipo y gana la clase más similar.

Figura 1. Clasificación por prototipos temáticos



Nota. Elaboración propia a partir del apartado 1.7.

Esta idea se relaciona con los clasificadores de centroides y con el algoritmo de Rocchio en recuperación de información, donde una clase o una consulta puede representarse como un vector prototípico dentro del espacio vectorial (Manning *et al.*, 2008). En esta unidad no se calcula un centroide a partir de muchos documentos etiquetados, sino un prototipo manual construido con palabras clave, aunque la lógica es parecida: comparar el texto nuevo con representaciones de clases.

La ventaja es la explicabilidad: si el prototipo jurídico contiene *ley*, *artículo*, *obligación* y *contrato*, el sistema tiene un criterio visible. La desventaja es que depende mucho de la calidad de esas listas, porque una clase pobremente representada será débil y una clase con palabras demasiado generales puede atraer textos que no le corresponden.

1.8. Preprocesamiento lingüístico con spaCy

El preprocesamiento convierte el texto bruto en unidades comparables. En esta unidad se usa spaCy, una biblioteca profesional de PLN, para tokenizar, lematizar, eliminar *stopwords* y filtrar *tokens* alfabéticos; su documentación describe el procesamiento lingüístico como una forma de pasar del texto crudo a un objeto `Doc` con anotaciones como *tokens*, lemas y atributos (Explosion AI, 2026).

Los pasos principales son:

- pasar el texto a minúsculas;
- dividirlo en *tokens*;
- conservar solo *tokens* alfabéticos;
- eliminar *stopwords*;
- conservar lemas;
- filtrar *tokens* demasiado cortos.

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

La lematización pesa mucho en español, porque permite agrupar variantes como *leyes* y *ley*, *contratos* y *contrato*, o *instalar* e *instalación*, en la medida en que el modelo pueda analizarlas correctamente. No es perfecta, pero ayuda a reducir la variación morfológica.

Eliminar *stopwords* también tiene un coste. Palabras como *por*, *en*, *de* o *que* suelen ser poco distintivas, pero algunas expresiones temáticas pueden contenerlas. En esta unidad la prioridad es que los vectores recojan señales léxicas de contenido; en tareas más finas, habría que revisar si ciertas palabras funcionales deben conservarse.

1.9. Explicabilidad: evidencias léxicas

La clasificación explicable busca que una persona pueda entender por qué se ha tomado una decisión. En los modelos complejos, técnicas como LIME (Ribeiro *et al.*, 2016) o SHAP (Lundberg y Lee, 2017) intentan explicar las predicciones mostrando qué rasgos han contribuido a ellas, pero esta unidad no las necesita, porque su modelo ya es interpretable por diseño.

La explicación se basa en evidencias léxicas:

- el texto se convierte en un vector;
- el prototipo ganador tiene su propio vector;
- se identifican términos compartidos;
- se ordenan por una contribución simple;
- se muestran al usuario.

Esto permite decir: «la clase técnica gana porque el texto comparte *dispositivo*, *puerto* y *controlador* con el prototipo técnico». La explicación no es perfecta, pero sí inspeccionable, y el alumnado puede discutir si esas palabras clave son adecuadas, si falta alguna, si sobran otras o si conviene usar TF-IDF.

1.10. Clasificación temática frente a modelos neuronales

Los modelos modernos de clasificación textual pueden lograr resultados muy superiores en muchos escenarios. Minaee *et al.* (2021) revisan más de 150 modelos de aprendizaje profundo para la clasificación textual, y Li *et al.* (2020) muestran la evolución desde los métodos superficiales hasta los enfoques profundos. Ahora bien, esos modelos requieren datos, entrenamiento y evaluación, y suelen ser menos transparentes.

El enfoque de PLN8 es útil cuando:

- hay pocos datos etiquetados;
- se necesita una primera solución rápida;
- importa justificar cada decisión;
- el dominio requiere auditoría humana;
- el objetivo es didáctico;
- las clases se pueden describir mediante palabras clave.

No es el enfoque ideal cuando:

- las clases dependen de significado implícito;
- hay mucha sinonimia;
- los textos son muy largos y heterogéneos;
- las categorías son abstractas;

- el vocabulario cambia mucho;
- se necesita alto rendimiento en producción.

Aprender este enfoque no supone retroceder frente a la IA moderna: es aprender la arquitectura conceptual que subyace a muchas técnicas más avanzadas, esto es, representación, ponderación, similitud y decisión.

1.11. Límites lingüísticos del método

Un clasificador por palabras clave y coseno funciona bien cuando las clases tienen un vocabulario característico, pero presenta varios límites:

Tabla 4. Límites lingüísticos del clasificador por palabras clave

Límite	Qué ocurre	Ejemplo
Sinonimia	No se detecta una idea si usa otra palabra	<i>normativa</i> frente a <i>ley</i>
Polisemia	Una palabra puede pertenecer a varias clases	<i>artículo</i> jurídico o <i>artículo</i> divulgativo
Textos híbridos	Dos clases comparten señales	divulgación técnica
Textos breves	Hay pocas evidencias	una frase muy corta
Metáfora e ironía	El sentido no está en palabras literales	estilo literario indirecto
Prototipos sesgados	La lista de palabras clave decide el criterio	clase técnica con más palabras clave que las demás

Para mejorar sin perder interpretabilidad se puede:

- ampliar palabras clave con variantes y sinónimos;
- usar TF-IDF en lugar de TF simple;
- conservar bigramas o expresiones;
- revisar *stopwords* por dominio;
- añadir umbrales de confianza;
- permitir etiqueta «indeterminado»;
- mostrar siempre evidencias y puntuaciones;
- validar con ejemplos humanos.

1.12. Qué significa «clasificar con responsabilidad»

Clasificar textos es algo más que aplicar una fórmula: supone crear una regla de decisión con consecuencias, como archivar un documento, priorizar una revisión, sugerir metadatos o enviar un texto a un equipo especializado. En instituciones públicas, bibliotecas digitales o entornos educativos, la transparencia importa.

Un informe responsable no debería decir simplemente «el sistema ha acertado». Debería indicar:

- qué clases estaban disponibles;
- qué palabras clave definían cada clase;
- cómo se preprocesaron los textos;
- qué medida de similitud se usó;
- qué puntuación obtuvo cada clase;
- qué evidencias léxicas apoyaron la decisión;
- qué límites tiene el resultado.

Esa forma de informar convierte el código en una herramienta lingüística auditada, cuya decisión no queda escondida: se puede revisar, corregir y mejorar.

2. Clasificación temática explicable con Python

2.1. Preparar el entorno

La unidad importa herramientas ligeras y comprensibles:

```
import re
from collections import Counter
from math import sqrt
from typing import List, Dict, Tuple
import pandas as pd
```

`re` permite trabajar con expresiones regulares, aunque en la parte principal el preprocesamiento se apoya en `spaCy`; `Counter` representa los vectores TF contando lemas; `sqrt` calcula las raíces cuadradas de la norma de los vectores en la similitud coseno; `List`, `Dict` y `Tuple` ayudan a documentar los tipos, y `pandas` permite construir tablas de resultados.

Después se importa spaCy y se carga el modelo de español:

```
import spacy
nlp = spacy.load("es_core_news_sm")
```

`nlp` es el *pipeline* lingüístico: cuando se aplica a un texto, devuelve un objeto con *tokens*, lemas, marcas de *stopword* y otros atributos.

2.2. Textos para clasificar

El código trabaja con una lista de textos:

```
texts_to_classify = [
    "Texto jurídico...",
    "Texto técnico...",
    "Texto divulgativo...",
    "Texto literario..."
]
```

Cada elemento es un documento que el sistema debe clasificar. En la actividad práctica, esos textos se sustituyen por fragmentos reales; en el manual no se resuelven esos casos, porque lo importante es comprender la estructura: una lista de documentos entra en el sistema y una tabla de clasificaciones sale al final.

En una versión con identificadores, la estructura puede ser:

```
("JUR_1", "Texto jurídico...")
```

Así se conserva un identificador junto con el texto, lo que facilita crear informes y rastrear decisiones.

2.3. Perfiles temáticos mediante palabras clave

Las clases se definen con un diccionario:

```
class_keywords = {
    "juridico": ["artículo", "ley", "cláusula", "contrato"],
    "técnico": ["instalar", "dispositivo", "puerto", "manual"],
    "divulgativo": ["explicar", "sencillo", "concepto", "introducción"],
    "literario": ["noche", "memoria", "horizonte", "metáfora"]
}
```

Cada clave del diccionario es una etiqueta de clase, y cada valor, una lista de palabras representativas. Esta estructura hace visible el criterio del sistema: si una palabra no está en el perfil, no podrá contribuir directamente a esa clase.

El diseño de las palabras clave es una tarea lingüística, y conviene que sean:

- específicas de la clase;
- suficientemente frecuentes en textos reales;
- no excesivamente generales;
- revisables por especialistas;
- equilibradas entre clases.

2.4. Preprocesamiento con spaCy

La función principal es:

```
def preprocess(text: str) -> List[str]:
    doc = nlp(text.lower())
    tokens = [
        tok.lemma_
        for tok in doc
        if tok.is_alpha and not tok.is_stop and len(tok) >= 2
    ]
    return tokens
```

Primero, `text.lower()` pasa el texto a minúsculas, y después `nlp(...)` aplica el *pipeline* de spaCy. La comprensión de listas recorre cada *token* y conserva solo los que cumplen tres condiciones:

- `tok.is_alpha`: el *token* está formado por letras;
- `not tok.is_stop`: no es una *stopword*;
- `len(tok) >= 2`: tiene al menos dos caracteres.

La forma que se guarda es `tok.lemma_`, es decir, el lema, lo que ayuda a comparar variantes morfológicas. La salida de la función es una lista de lemas, y no un texto.

2.5. Vectorización TF con Counter

La función de vectorización es mínima:

```
def tf_vector(tokens: List[str]) -> Counter:
    return Counter(tokens)
```

Recibe una lista de *tokens* o lemas y devuelve un contador: si el texto produce `["ley", "contrato", "ley"]`, el vector será `Counter({"ley": 2, "contrato": 1})`.

Este `Counter` funciona como vector disperso, porque solo guarda los términos que aparecen; los ausentes se comportan como si tuvieran frecuencia cero.

Este diseño resulta muy cómodo para textos: no hace falta crear una lista gigantesca con todas las palabras posibles, porque solo se almacenan las presentes.

2.6. Similitud coseno en Python

La función de similitud coseno es:

```
def cosine_similarity(v1: Counter, v2: Counter) -> float:
    common = set(v1.keys()) & set(v2.keys())
    dot = sum(v1[t] * v2[t] for t in common)
    norm1 = sqrt(sum(v**2 for v in v1.values()))
    norm2 = sqrt(sum(v**2 for v in v2.values()))
    if norm1 == 0 or norm2 == 0:
        return 0.0
    return dot / (norm1 * norm2)
```

`common` calcula los términos compartidos entre los dos vectores, y el operador `&` obtiene la intersección de conjuntos.

`dot` calcula el producto escalar: para cada término común, multiplica la frecuencia en el primer vector por la frecuencia en el segundo y suma.

`norm1` y `norm2` calculan la longitud de cada vector: elevan cada frecuencia al cuadrado, suman y aplican la raíz cuadrada.

Si alguna norma es cero, uno de los vectores está vacío, y en ese caso la similitud devuelve `0.0` para evitar una división entre cero.

Con este tipo de vectores, que no tienen valores negativos, la función devuelve un número entre 0 y 1.

2.7. Construir prototipos temáticos

Las palabras clave de cada clase se convierten en prototipos vectoriales:

```
def build_class_prototypes(class_keywords: dict) -> dict:
    prototypes = {}
    for label, keywords in class_keywords.items():
        tokens = []
        for kw in keywords:
            tokens.extend(preprocess(kw))
        prototypes[label] = tf_vector(tokens)
    return prototypes
```

La función recorre cada etiqueta y su lista de palabras clave; cada palabra clave se preprocesa y sus lemas se añaden a una lista, que después se convierte en vector TF.

El resultado es un diccionario de prototipos:

```
prototypes = {
    "jurídico": Counter(...),
    "técnico": Counter(...),
    "divulgativo": Counter(...),
    "literario": Counter(...)
}
```

Cada clase es ahora, además de una lista de palabras, un vector comparable con cualquier texto.

2.8. Evidencias léxicas

La función de evidencias identifica qué lemas del texto aparecen también en el prototipo ganador:

```
def top_evidence_terms(text_vec: Counter, proto_vec: Counter, top_k: int = 8):
    contrib = []
    for t, f in text_vec.items():
        if t in proto_vec:
            contrib.append((t, f * proto_vec[t]))
    contrib.sort(key=lambda x: x[1], reverse=True)
    return contrib[:top_k]
```

La contribución simple se calcula como la frecuencia del término en el texto multiplicada por su frecuencia en el prototipo, de modo que una palabra que aparece varias veces y además está en el prototipo tendrá más peso.

La salida es una lista de pares:

```
[("ley", 2), ("contrato", 1)]
```

Esto permite presentar una evidencia legible: la explicación no dice «el modelo pensó esto», sino «la coincidencia léxica que más apoya esta clase es esta».

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

2.9. Clasificar un texto

La función `classify_text` reúne todo el *pipeline*:

```
def classify_text(text: str, prototypes: dict) -> dict:
    tokens = preprocess(text)
    vec = tf_vector(tokens)
    scores = {
        label: cosine_similarity(vec, proto)
        for label, proto in prototypes.items()
    }
    best_label = max(scores, key=scores.get)
    evidence = top_evidence_terms(vec, prototypes[best_label])
    return {
        "text": text,
        "tokens": tokens,
        "predicted_label": best_label,
        "scores": scores,
        "evidence": evidence
    }
```

Primero preprocesa el texto, después lo vectoriza y luego calcula una similitud por cada prototipo; `scores` es un diccionario en el que cada clase tiene su puntuación.

`max(scores, key=scores.get)` devuelve la etiqueta con la puntuación más alta, y por último se calculan las evidencias frente al prototipo ganador.

La función devuelve un diccionario completo, y no solo la etiqueta, una buena práctica que conserva datos, puntuaciones y explicación.

2.10. Clasificar varios textos y crear una tabla

Para aplicar el sistema a todos los textos se usa una comprensión de listas:

```
results = [classify_text(t, prototypes) for t in texts_to_classify]
```

Después se construye una tabla:

```
rows = []
for r in results:
    row = {
        "predicted_label": r["predicted_label"],
        "text": r["text"]
    }
    for label, score in r["scores"].items():
        row[f"sim_{label}"] = score
    rows.append(row)

df_scores = pd.DataFrame(rows)
```

Cada fila contiene el texto, la clase predicha y las puntuaciones de similitud. Las columnas `sim_jurídico`, `sim_técnico`, `sim_divulgativo` y `sim_literario` permiten comparar por qué gana una clase y no otra.

La tabla forma parte de la explicabilidad, porque además del resultado final muestra las alternativas y las diferencias de puntuación.

2.11. Informe de clasificación

La función `print_report` convierte los resultados en una salida narrativa:

```
def print_report(results):
    for i, r in enumerate(results, 1):
        print(f"TEXTO #{i}")
        print(r["text"])
        print(f"Clase predicha: {r['predicted_label']}")
        for label, score in sorted(r["scores"].items(), key=lambda x: x[1],
reverse=True):
            print(f"{label}: {score:.3f}")
        print("Evidencias:", r["evidence"])
```

El informe ordena las similitudes de mayor a menor y muestra las evidencias, algo más útil para una persona no técnica que una tabla de números sin explicación.

En un entorno profesional, este informe podría convertirse en una ficha de revisión: clase sugerida, confianza aproximada, evidencias y comentario humano.

2.12. Dónde entraría TF-IDF

El cuaderno principal utiliza TF simple. Una versión ampliada podría sustituir o complementar esos vectores con TF-IDF. En scikit-learn, el patrón general sería:

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(textos)
```

La idea sería que las palabras demasiado comunes pesaran menos: por ejemplo, si *sistema* aparece en textos técnicos y divulgativos, TF-IDF puede reducir su peso frente a palabras más distintivas. Ahora bien, en un sistema de prototipos manuales hay que decidir con cuidado sobre qué colección se calcula el IDF (documentos reales, prototipos, textos de entrenamiento o una mezcla), porque esa decisión cambia las ponderaciones.

Para esta unidad, basta con entender la diferencia:

Tabla 5. TF, IDF y TF-IDF

Representación	Qué mide	Ventaja	Límite
TF	Frecuencia dentro del texto	Muy interpretable	No penaliza palabras comunes
IDF	Especificidad en la colección	Reduce peso de términos generales	Depende del corpus de referencia
TF-IDF	Frecuencia ponderada por especificidad	Más útil en colecciones grandes	Menos transparente que TF puro

2.13. Cómo leer los resultados antes de la práctica

Cuando ejecutes el cuaderno, no mires solo la etiqueta; observa también:

- qué *tokens* produjo `preprocess`;
- qué palabras clave definían cada prototipo;
- qué similitud obtuvo cada clase;
- cuánto margen hay entre la clase ganadora y la segunda;
- qué evidencias aparecen;
- qué evidencias faltan;
- si el texto podría ser híbrido;
- si una palabra clave demasiado general está influyendo demasiado.

Una clasificación responsable se formula así: «el sistema asigna la clase *técnico* porque el vector del texto comparte estos lemas con el prototipo técnico y obtiene mayor similitud que las demás clases», una frase más precisa que «el texto es técnico».

Errores frecuentes de aprendizaje

- Confundir palabras clave con significado completo del texto.
- Pensar que TF-IDF entiende semántica, cuando solo pondera términos.
- Interpretar coseno alto como comprensión profunda.
- Olvidar que *bag-of-words* pierde el orden de las palabras.
- Creer que el prototipo temático es neutral: lo diseña una persona.
- Comparar puntuaciones sin mirar evidencias.
- No revisar si las *stopwords* eliminadas eran importantes.
- Usar palabras clave demasiado generales, como *sistema* o *información*, sin control.
- No distinguir textos híbridos de errores del clasificador.
- Presentar la etiqueta predicha sin explicar el criterio.

Síntesis final

Esta unidad presenta la clasificación temática explicable desde la base: palabras clave, vectores simples, frecuencia, similitud coseno y evidencias léxicas. En lugar de ocultar la decisión dentro de un modelo complejo, se aprende cómo se construye una decisión automática revisable.

El marco teórico relaciona el cuaderno con el modelo de espacio vectorial, TF, IDF, TF-IDF, los prototipos temáticos, la clasificación textual y la interpretabilidad, conceptos que forman una base imprescindible para entender métodos posteriores más sofisticados, incluidos los clasificadores supervisados y los modelos neuronales.

La parte de Python muestra un *pipeline* completo: cargar spaCy, preprocesar textos, convertir lemas en vectores TF, construir prototipos por clase, calcular similitud coseno, elegir la clase con mayor puntuación y mostrar evidencias. Cada bloque puede explicarse lingüísticamente.

La idea principal es que clasificar va más allá de etiquetar: clasificar con responsabilidad significa definir criterios, mostrar puntuaciones, ofrecer evidencias y permitir la revisión humana.

Cierre narrativo

Al terminar la simulación, el Centro Atlas de Documentación Textual recibe un prototipo transparente en lugar de una caja negra: puede revisar las palabras clave de cada clase, ver cómo se procesan los textos, observar qué similitud tiene cada documento con cada perfil y comprobar qué lemas explican la decisión. Ese es el valor profesional de esta unidad. Una lingüista computacional hace mucho más que ejecutar código, porque diseña criterios de clasificación, detecta ambigüedades, interpreta evidencias y comunica límites; si un documento se clasifica como jurídico, técnico, divulgativo o literario, la decisión debe poder contarse con palabras.

En clase se trabajarán los casos concretos. Este manual prepara el terreno: cuando ejecutes el cuaderno, ya sabrás qué significa cada vector, por qué importa cada palabra clave y qué está midiendo realmente el coseno.

Para saber más: Preparación previa a la práctica

Ali, M. (2024, 3 de mayo). Clasificación de textos en Python. DataCamp. <https://www.datacamp.com/es/tutorial/text-classification-python>

Alura Latam. (s. f.). ¿Qué es Bag of Words? #AluraMás [vídeo]. YouTube. <https://www.youtube.com/watch?v=NKy59utXjcg&t=194s>

Barnard, J. (s. f.). ¿Qué son las incrustaciones de palabras? IBM. <https://www.ibm.com/es-es/think/topics/word-embeddings>

Gutiérrez, O. [Codigo Maquina]. (s. f.). TF-IDF: Vectorización de Documentos para NLP con Python [vídeo]. YouTube. <https://www.youtube.com/watch?v=4QsG4oNcGpM&t=583s>

Krantz, T. y Jonker, A. (s. f.). ¿Qué es la similitud de coseno? IBM. <https://www.ibm.com/es-es/think/topics/cosine-similarity>

Murel, J. y Kavlakoglu, E. (s. f.). ¿Qué es la bolsa de palabras? IBM. <https://www.ibm.com/es-es/think/topics/bag-of-words>

Starmer, J. [StatQuest with Josh Starmer]. (s. f.). *Cosine similarity, clearly explained!!!* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=e9U0QAFbflI>

Referencias

- Explosion AI. (2026). *Linguistic features*. spaCy Usage Documentation. <https://spacy.io/usage/linguistic-features>
- Gasparetto, A., Marcuzzo, M., Zangari, A. y Albarelli, A. (2022). A survey on text classification algorithms: From text to predictions. *Information*, 13(2), artículo 83. <https://doi.org/10.3390/info13020083>
- Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L. y Brown, D. (2019). Text classification algorithms: A survey. *Information*, 10(4), artículo 150. <https://doi.org/10.3390/info10040150>
- Li, Q., Peng, H., Li, J., Xia, C., Yang, R., Sun, L., Yu, P. S. y He, L. (2020). *A survey on text classification: From shallow to deep learning* [Prepublicación]. arXiv. <https://arxiv.org/abs/2008.00364>
- Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.
- Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. <https://nlp.stanford.edu/IR-book/>
- Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, M. y Gao, J. (2021). Deep learning-based text classification: A comprehensive review. *ACM Computing Surveys*, 54(3), artículo 62. <https://doi.org/10.1145/3439726>
- Papagiannopoulou, E. y Tsoumakas, G. (2020). A review of keyphrase extraction. *WIREs Data Mining and Knowledge Discovery*, 10(2), artículo e1339. <https://doi.org/10.1002/widm.1339>
- Ribeiro, M. T., Singh, S. y Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144). ACM. <https://doi.org/10.1145/2939672.2939778>
- Robertson, S. (2004). Understanding inverse document frequency: On theoretical arguments for IDF. *Journal of Documentation*, 60(5), 503–520. <https://doi.org/10.1108/00220410410560582>
- Rose, S., Engel, D., Cramer, N. y Cowley, W. (2010). Automatic keyword extraction from individual documents. En M. W. Berry y J. Kogan (Eds.), *Text mining: Applications and theory* (pp. 1–20). Wiley. <https://doi.org/10.1002/9780470689646.ch1>
- Salton, G., Wong, A. y Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 613–620. <https://doi.org/10.1145/361219.361220>
- scikit-learn developers. (2026). *TfidfVectorizer*. scikit-learn Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1), 1–47. <https://doi.org/10.1145/505282.505283>
- Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21. <https://doi.org/10.1108/eb026526>